```
// File: bin\.gitkeep
```

```
// File: bin\boost.js
#!/usr/bin/env node

import { program } from "commander";
import { runScan } from "../src/commands/scan.js";
import { runDeps } from "../src/commands/deps.js";
import { runScore } from "../src/commands/score.js";
import cleanCommand from "../src/commands/clean.js";
import securityCommand from "../src/commands/security.js";

program.name("boost").description("Project health toolkit").version("1.0.0");

program
  .command("scan")
  .description(
    "Scan for unused files, large files, duplicate code, unused assets,
console.logs",
  )
  .action(runScan);

program
  .command("deps")
  .description("Analyse dependencies — unused, heavy, and alternatives")
  .action(runDeps);

program
  .command("clean")
  .description("Remove temporary files and empty folders")
  .action(cleanCommand);

program
  .command("security")
  .description("Check for committed secrets and vulnerabilities")
  .action(securityCommand);

program
  .command("score")
  .description("Show project health score out of 100")
  .action(runScore);

// No subcommand !’ run everything
if (process.argv.length === 2) {
  await runScan();
  await runDeps();
  await cleanCommand();
  await securityCommand();
  await runScore();
} else {
  program.parse();
}
```

```
// File: package.json
{
  "name": "proj-boost",
  "version": "1.0.0",
  "description": "Project health toolkit",
  "type": "module",
  "main": "./src/index.js",
  "bin": {
    "boost": "bin/boost.js"
  },
  "scripts": {
    "test": "vitest"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Bhumi-Garg/npm-boost.git"
  },
  "keywords": ["cli", "toolkit", "cleanup", "health"],
  "author": "Bhumi-Garg",
  "license": "ISC",
  "bugs": {
    "url": "https://github.com/Bhumi-Garg/npm-boost/issues"
  },
  "homepage": "https://github.com/Bhumi-Garg/npm-boost#readme",
  "dependencies": {
    "chalk": "^5.6.2",
    "commander": "^14.0.3",
    "depcheck": "^1.4.7",
    "fast-glob": "^3.3.3",
    "fs-extra": "^11.3.3",
    "ignore": "^7.0.5",
    "jscpd": "^4.0.8",
    "ora": "^9.3.0"
  },
  "devDependencies": {
    "eslint": "^9.39.3",
    "prettier": "^3.8.1",
    "vitest": "^3.2.4"
  }
}
```

// File: README.md

proj-boost

> A CLI toolkit to analyze, clean, and score the health of any JavaScript/TypeScript project — in seconds.
>

Ø=Üæ Installation

```bash
npm install -g proj-boost
```

&¡ Quick Start

Navigate to any JavaScript/TypeScript project and run:

```bash
boost
```

This runs all checks and shows a full health report.

Ø=Þàþ Commands

Command	Description
`boost`	Run everything
`boost scan`	Scan for code quality issues
`boost clean`	Remove temp files and empty folders
`boost deps`	Audit dependencies
`boost score`	Show project health score
`boost security`	Run security checks

Ø=Ý boost scan

Scans your project for common code quality issues.

```bash
boost scan
```

Checks:

- **Unused Files** — files not imported from your entry point
- **Large Files** — files above 500kb (configurable)
- **Duplicate Code** — repeated code blocks across files
- **Unused Assets** — images/fonts not referenced in code
- **Console Logs** — leftover `console.log` statements

Example output:

```
Ø=Ý  boost scan — Project Analysis

Ø=ÜË Scan Results
&  Unused Files      — 2 unused file(s) found
   !’ src/helpers/oldUtil.js
   !’ src/components/TestModal.jsx
'  Large Files       — No large files found
'  Duplicate Code    — No duplicate code found
'  Unused Assets     — No unused assets found
&  Console Logs      — 5 console.log(s) found
   !’ src/App.jsx:12  console.log("debug")
```

Ø>Ýù boost clean

Removes temporary files and empty folders from your project.

```bash
boost clean
```

Removes:

- Temporary files (`.tmp`, `.cache`, `.DS_Store`, `Thumbs.db`)
- Empty folders

Ø=Üæ boost deps

Audits your project dependencies.

```bash
boost deps
```

Checks:

- **Unused packages** — packages in `package.json` not used in code
- **Heavy packages** — packages with large install size (over 100KB gzip)
- **Lighter alternatives** — suggests smaller replacements

Example output:

```
Dependency analysis

&  Unused packages     — 2 unused package(s) found
   !’ lodash
   !’ moment
&  Heavy packages      — 1 package(s) over 100 KB gzip
   !’ eslint  347.2 KB gzip / 1309.1 KB raw
&  Lighter alternatives — 1 package(s) have lighter alternatives
   !’ chalk !’ picocolors or kleur
      ~10x smaller with near-identical API
```

Ø=ÜÊ boost score

Shows an overall health score for your project out of 100 with a letter grade.

```bash
boost score
```

Categories:

Category	Weight
Code Quality	40 pts
Security	30 pts
Dependencies	20 pts
Cleanliness	10 pts

Example output:

```
Project health score
'  All checks complete

  79/100    Grade C

  %^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%‘%‘%‘%‘%‘%‘%‘%‘   65%  Code Quality    26/40 pts
  %^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^  100%  Security        30/30 pts
  %^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%‘%‘%‘%‘%‘%‘%‘%‘   65%  Dependencies    13/20 pts
  %^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^%^  100%  Cleanliness     10/10 pts

Check breakdown

  &   Unused Files       — 2 unused file(s) found       [8/10]
  '   Large Files        — No large files found          [8/8]
  '   Duplicate Code     — No duplicate code found       [10/10]
  '   Unused Assets      — No unused assets found        [6/6]
  &   Console Logs       — 5 console.log(s) found        [1/6]
  '   .env Files         — No committed .env files       [12/12]
  '   Hardcoded Secrets  — No hardcoded secrets found    [12/12]
  '   npm Vulnerabilities — No vulnerabilities found     [6/6]
```

Ø=Ý boost security

Checks your project for security issues.

```bash
boost security
```

Checks:

- **.env files committed** — detects `.env` files tracked by git
- **Hardcoded secrets** — scans for API keys, tokens, passwords in code
- **npm vulnerabilities** — wraps `npm audit` and parses results

&™þ Configuration

Create a `boost.config.js` in your project root to customize behavior:

```js
export default {
  scan: {
    largeFileThreshold: "500kb", // flag files above this size
    ignore: ["dist/", "node_modules/"], // folders to skip
    extensions: [".js", ".ts", ".jsx", ".tsx", ".vue"],
  },
  clean: {
    tempPatterns: ["**/*.tmp", "**/.DS_Store"],
  },
  deps: {
    checkBundleSize: true,
  },
  security: {
    scanPatterns: ["**/*.js", "**/*.ts"],
  },
};
```

Supported Project Types

- React / Next.js
- Node.js / Express
- Vue / Nuxt
- Plain JavaScript / TypeScript

Requirements

- Node.js `>= 18.0.0`
- npm `>= 8.0.0`

Contributing

Pull requests are welcome! For major changes, please open an issue first.

```bash
git clone https://github.com/Bhumi-Garg/npm-boost.git
cd boost-cli
npm install
npm link
boost scan
```

```
// File: src\cleaners\.gitkeep
```

```
// File: src\cleaners\emptyFolders.js
import fs from "fs-extra";
import path from "path";
import { loadConfig } from "../utils/config.js";

async function isEmpty(dir) {
  const files = await fs.readdir(dir);
  return files.length === 0;
}

async function walk(dir, removed = []) {
  const entries = await fs.readdir(dir);

  for (const entry of entries) {
    const fullPath = path.join(dir, entry);
    const stat = await fs.stat(fullPath);

    if (stat.isDirectory()) {
      await walk(fullPath, removed);

      if (await isEmpty(fullPath)) {
        await fs.remove(fullPath);
        removed.push(path.relative(process.cwd(), fullPath));
      }
    }
  }

  return removed;
}

export default async function emptyFolders() {
  try {
    await loadConfig(); // keeps pattern consistent even if unused

    const removed = await walk(process.cwd());

    return {
      label: "Empty Folders",
      status: removed.length > 0 ? "warn" : "ok",
      count: removed.length,
      data: removed,
      summary:
        removed.length > 0
          ? `${removed.length} empty folders removed`
          : "No empty folders found",
    };
  } catch (error) {
    return {
      label: "Empty Folders",
      status: "error",
      count: 0,
      data: [],
      summary: "Failed to clean empty folders",
    };
  }
}
```

```
// File: src\cleaners\tempFiles.js
import fg from "fast-glob";
import fs from "fs-extra";
import path from "path";
import { loadConfig } from "../utils/config.js";

export default async function tempFiles() {
  try {
    const config = await loadConfig();
    const patterns = config.clean?.tempPatterns || [];

    const files = await fg(patterns, {
      dot: true,
      absolute: true,
    });

    let removed = [];

    for (const file of files) {
      try {
        await fs.remove(file);
        removed.push(path.relative(process.cwd(), file));
      } catch (err) {
        // skip but continue
      }
    }

    return {
      label: "Temporary Files",
      status: removed.length > 0 ? "warn" : "ok",
      count: removed.length,
      data: removed,
      summary:
        removed.length > 0
          ? `${removed.length} temporary files removed`
          : "No temporary files found",
    };
  } catch (error) {
    return {
      label: "Temporary Files",
      status: "error",
      count: 0,
      data: [],
      summary: "Failed to clean temporary files",
    };
  }
}
```

```
// File: src\commands\.gitkeep
```

```
// File: src\commands\clean.js
import tempFiles from "../cleaners/tempFiles.js";
import emptyFolders from "../cleaners/emptyFolders.js";

import { logger } from "../utils/logger.js";
import { startSpinner, stopSpinner } from "../utils/spinner.js";

export default async function cleanCommand() {
  try {
    startSpinner("Cleaning project...");

    const results = [];

    const tempResult = await tempFiles();
    results.push(tempResult);

    const folderResult = await emptyFolders();
    results.push(folderResult);

    stopSpinner(true, "Clean complete");

    logger.title("Clean Results");

    for (const r of results) {
      logger.result(r);
    }

    return results;
  } catch (err) {
    stopSpinner(false, "Clean command failed");
    logger.error(err.message);
  }
}
```

```
// File: src\commands\deps.js
import chalk from "chalk";
import { checkUnusedPackages } from "../deps/unusedPackages.js";
import { checkHeavyPackages } from "../deps/heavyPackages.js";
import { checkLighterAlternatives } from "../deps/lighterAlternatives.js";
import { logger } from "../utils/logger.js";
import { startSpinner, stopSpinner, updateSpinner } from "../utils/
spinner.js";

function printUnused(result) {
  logger.result({
    ...result,
    data: result.data, // already strings (package names)
  });
}

function printHeavy(result) {
  if (result.status !== "warn") {
    logger.result(result);
    return;
  }

  logger.warn(chalk.yellow(result.label) + chalk.dim(` — ${result.summary}`));
  result.data.forEach((item) => {
    console.log(
      chalk.dim("   !’"),
      chalk.white(item.name),
      chalk.dim(`${item.gzipKb} KB gzip`),
      chalk.dim(`/ ${item.sizeKb} KB raw`),
    );
  });
}

function printAlternatives(result) {
  if (result.status !== "warn") {
    logger.result(result);
    return;
  }

  logger.warn(chalk.yellow(result.label) + chalk.dim(` — ${result.summary}`));
  result.data.forEach((item) => {
    console.log(
      chalk.dim("   !’"),
      chalk.white(item.name),
      chalk.dim("!’"),
      chalk.cyan(item.suggestion),
    );
    console.log(chalk.dim(`      ${item.reason}`));
  });
}

export async function runDeps() {
  logger.title("Dependency analysis");

  startSpinner("Checking unused packages...");
  const unused = await checkUnusedPackages();
  updateSpinner("Checking bundle sizes...");
  const heavy = await checkHeavyPackages();
  updateSpinner("Looking for lighter alternatives...");
  const alternatives = await checkLighterAlternatives();
  stopSpinner(true, "Dependency analysis complete");
```

```
  logger.blank();
  printUnused(unused);
  printHeavy(heavy);
  printAlternatives(alternatives);
  logger.blank();

  const totalIssues = unused.count + heavy.count + alternatives.count;
  if (totalIssues === 0) {
    logger.success("All dependency checks passed");
  } else {
    logger.dim(`${totalIssues} issue(s) found across dependency checks`);
  }

  return [unused, heavy, alternatives];
}
```

```
// File: src\commands\scan.js
import { scanConsoleLogs } from "../scanners/consoleLogs.js";
import { scanLargeFiles } from "../scanners/largeFiles.js";
import { scanUnusedAssets } from "../scanners/unusedAssets.js";
import { scanUnusedFiles } from "../scanners/unusedFiles.js";
import { scanDuplicateCode } from "../scanners/duplicateCode.js";
import { logger } from "../utils/logger.js";
import { startSpinner, stopSpinner } from "../utils/spinner.js";

export async function runScan() {
  logger.title("boost scan — Project Analysis");

  const checks = [
    { label: "Scanning for unused files...", fn: scanUnusedFiles },
    { label: "Scanning for large files...", fn: scanLargeFiles },
    { label: "Scanning for duplicate code...", fn: scanDuplicateCode },
    { label: "Scanning for unused assets...", fn: scanUnusedAssets },
    { label: "Scanning for console.logs...", fn: scanConsoleLogs },
  ];

  const results = [];

  for (const check of checks) {
    startSpinner(check.label);
    try {
      const result = await check.fn();
      stopSpinner(true, check.label.replace("...", " ' "));
      results.push(result);
    } catch (err) {
      stopSpinner(false, check.label.replace("...", " ' "));
      results.push({
        label: check.label,
        status: "error",
        count: 0,
        data: [],
        summary: err.message,
      });
    }
  }

  logger.blank();
  logger.title("Scan Results");
  results.forEach((r) => logger.result(r));
  logger.blank();

  return results;
}
```

```
// File: src\commands\score.js
import chalk from "chalk";
import { computeHealthScore } from "../scorer/healthScore.js";
import { logger } from "../utils/logger.js";
import { startSpinner, stopSpinner } from "../utils/spinner.js";

// % % %  Helpers % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %

function scoreColor(score) {
  if (score >= 90) return chalk.green;
  if (score >= 70) return chalk.cyan;
  if (score >= 50) return chalk.yellow;
  return chalk.red;
}

function gradeColor(grade) {
  if (grade === "A") return chalk.green;
  if (grade === "B") return chalk.cyan;
  if (grade === "C") return chalk.yellow;
  if (grade === "D") return chalk.red;
  return chalk.bgRed.white;
}

function statusIcon(status) {
  if (status === "ok") return chalk.green("' ");
  if (status === "warn") return chalk.yellow("& ");
  if (status === "error") return chalk.red("' ");
  return chalk.dim("–");
}

function bar(pct, width = 24) {
  const filled = Math.round((pct / 100) * width);
  const empty = width - filled;
  const col = scoreColor(pct);
  return col("%ˆ".repeat(filled)) + chalk.dim("%‘".repeat(empty));
}

function formatItem(item) {
  if (typeof item === "string") return item;
  if (item.name && item.suggestion)
    return `${item.name}  !’  ${item.suggestion}`;
  if (item.name && item.gzipKb) return `${item.name}  ${item.gzipKb} KB gzip`;
  return JSON.stringify(item);
}

// % % %  Sections % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %

function printScoreCard(score, grade) {
  const col = scoreColor(score);
  const gcol = gradeColor(grade);
  console.log("");
  console.log(
    "  " +
      col(chalk.bold(`${score}`)) +
      chalk.dim("/100") +
      "    " +
      gcol(chalk.bold(`Grade ${grade}`)),
  );
  console.log("");
}
```

```
function printCategories(categories) {
  for (const cat of Object.values(categories)) {
    console.log(
      "  " +
        bar(cat.pct) +
        "  " +
        scoreColor(cat.pct)(`${String(cat.pct).padStart(3)}%`) +
        "  " +
        chalk.white(cat.label) +
        chalk.dim(`  ${cat.earned}/${cat.maxPts} pts`),
    );
  }
  console.log("");
}

function printChecks(checks) {
  logger.title("Check breakdown");
  console.log("");

  for (const [, val] of Object.entries(checks)) {
    const { result, pts, maxPts } = val;

    console.log(
      "  " +
        statusIcon(result.status) +
        "  " +
        chalk.white(result.label) +
        chalk.dim(`  — ${result.summary}`) +
        chalk.dim(`  [${pts}/${maxPts}]`),
    );

    if (result.status === "warn" && result.data?.length) {
      const preview = result.data.slice(0, 3);
      preview.forEach((item) => {
        console.log(chalk.dim(`       • ${formatItem(item)}`));
      });
      if (result.data.length > 3) {
        console.log(chalk.dim(`       … and ${result.data.length - 3} more`));
      }
    }
  }

  console.log("");
}

function printFooter(score) {
  if (score >= 90) {
    logger.success("Excellent project health!");
  } else if (score >= 70) {
    logger.info("Good shape — a few things to tighten up.");
  } else if (score >= 50) {
    logger.warn("Some issues need attention.");
  } else {
    logger.error("Project needs significant work.");
  }
  console.log("");
}

// % % %  Command entry % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %

export async function runScore() {
```

```
  logger.title("Project health score");
  startSpinner("Running all checks...");

  let health;
  try {
    health = await computeHealthScore();
  } catch (err) {
    stopSpinner(false, "Health score failed");
    logger.error(err.message);
    return;
  }

  stopSpinner(true, "All checks complete");

  printScoreCard(health.score, health.grade);
  printCategories(health.categories);
  printChecks(health.checks);
  printFooter(health.score);

  return health;
}
```

```
// File: src\commands\security.js
import envCommitted from "../security/envCommitted.js";
import hardcodedKeys from "../security/hardcodedKeys.js";
import auditWrapper from "../security/auditWrapper.js";

import { logger } from "../utils/logger.js";
import { startSpinner, stopSpinner } from "../utils/spinner.js";

export default async function securityCommand() {
  try {
    startSpinner("Running security checks...");

    const results = [];

    const envResult = await envCommitted();
    results.push(envResult);

    const secretResult = await hardcodedKeys();
    results.push(secretResult);

    const auditResult = await auditWrapper();
    results.push(auditResult);

    stopSpinner(true, "Security scan complete");

    logger.title("Security Results");

    for (const r of results) {
      logger.result(r);
    }

    return results;
  } catch (err) {
    stopSpinner(false, "Security scan failed");
    logger.error(err.message);
  }
}
```

```
// File: src\deps\.gitkeep
```

```
// File: src\deps\heavyPackages.js
import { readFile } from "fs/promises";
import { join } from "path";
import { loadConfig } from "../utils/config.js";

const BUNDLEPHOBIA_API = "https://bundlephobia.com/api/size?package=";
const DEFAULT_THRESHOLD_KB = 100;

async function fetchBundleSize(pkg) {
  try {
    const res = await fetch(`${BUNDLEPHOBIA_API}${encodeURIComponent(pkg)}`, {
      signal: AbortSignal.timeout(8000),
    });
    if (!res.ok) return null;
    const json = await res.json();
    if (!json.gzip) return null;
    return {
      name: pkg,
      gzipKb: parseFloat((json.gzip / 1024).toFixed(1)),
      sizeKb: parseFloat((json.size / 1024).toFixed(1)),
    };
  } catch {
    return null;
  }
}

export async function checkHeavyPackages() {
  const config = await loadConfig();

  if (config.deps?.checkBundleSize === false) {
    return {
      label: "Heavy packages",
      status: "ok",
      count: 0,
      data: [],
      summary: "Bundle size check disabled in config",
    };
  }

  try {
    const pkgRaw = await readFile(join(process.cwd(), "package.json"),
"utf8");
    const pkg = JSON.parse(pkgRaw);
    const deps = Object.keys({ ...pkg.dependencies, ...pkg.devDependencies });

    const thresholdKb = config.deps?.heavyThresholdKb ?? DEFAULT_THRESHOLD_KB;

    // Fetch in batches of 5 to avoid hammering bundlephobia
    const results = [];
    for (let i = 0; i < deps.length; i += 5) {
      const batch = deps.slice(i, i + 5);
      const sizes = await Promise.all(batch.map(fetchBundleSize));
      results.push(...sizes.filter(Boolean));
    }

    const heavy = results
      .filter((p) => p.gzipKb > thresholdKb)
      .sort((a, b) => b.gzipKb - a.gzipKb);

    return {
      label: "Heavy packages",
```

```
      status: heavy.length > 0 ? "warn" : "ok",
      count: heavy.length,
      data: heavy,
      summary:
        heavy.length > 0
          ? `${heavy.length} package(s) over ${thresholdKb} KB gzip`
          : `All packages under ${thresholdKb} KB gzip`,
    };
  } catch (err) {
    return {
      label: "Heavy packages",
      status: "error",
      count: 0,
      data: [],
      summary: `Bundle size check failed: ${err.message}`,
    };
  }
}
```

```
// File: src\deps\lighterAlternatives.js
import { readFile } from "fs/promises";
import { join } from "path";

const ALTERNATIVES_MAP = {
  moment: {
    suggestion: "date-fns or dayjs",
    reason: "Much smaller and tree-shakable",
  },
  lodash: {
    suggestion: "lodash-es or native ES",
    reason: "Use partial imports or built-in array methods",
  },
  underscore: {
    suggestion: "lodash-es or native ES",
    reason: "Lodash is a strict superset with better tree-shaking",
  },
  request: {
    suggestion: "node-fetch or got",
    reason: "request is officially deprecated",
  },
  axios: {
    suggestion: "native fetch (Node 18+)",
    reason: "fetch is built into Node — no extra dep needed",
  },
  "node-fetch": {
    suggestion: "native fetch (Node 18+)",
    reason: "fetch is built into Node — no extra dep needed",
  },
  bluebird: {
    suggestion: "native Promise",
    reason: "Native Promises are fast and fully featured now",
  },
  "node-uuid": {
    suggestion: "crypto.randomUUID()",
    reason: "Built into Node.js >= 14.17, zero deps",
  },
  uuid: {
    suggestion: "crypto.randomUUID()",
    reason: "Built into Node.js >= 14.17, zero deps",
  },
  mkdirp: {
    suggestion: "fs.mkdirSync({ recursive })",
    reason: "Built into Node.js, no package needed",
  },
  rimraf: {
    suggestion: "fs.rmSync({ recursive })",
    reason: "Built into Node.js >= 14.14",
  },
  glob: {
    suggestion: "node:fs glob (Node 22+)",
    reason: "Built-in glob available in Node 22+",
  },
  chalk: {
    suggestion: "picocolors or kleur",
    reason: "~10x smaller with near-identical API",
  },
  colors: {
    suggestion: "picocolors or kleur",
    reason: "colors has had supply-chain issues; picocolors is safer",
  },
```

```
  sprintf: {
    suggestion: "native template literals",
    reason: "Template literals cover most sprintf use cases",
  },
  "is-array": {
    suggestion: "Array.isArray()",
    reason: "Built-in JS, no package needed",
  },
};

export async function checkLighterAlternatives() {
  try {
    const pkgRaw = await readFile(join(process.cwd(), "package.json"),
"utf8");
    const pkg = JSON.parse(pkgRaw);
    const deps = Object.keys({ ...pkg.dependencies, ...pkg.devDependencies });

    const found = deps
      .filter((d) => ALTERNATIVES_MAP[d])
      .map((d) => ({ name: d, ...ALTERNATIVES_MAP[d] }));

    return {
      label: "Lighter alternatives",
      status: found.length > 0 ? "warn" : "ok",
      count: found.length,
      data: found,
      summary:
        found.length > 0
          ? `${found.length} package(s) have lighter alternatives`
          : "No obvious replacements found",
    };
  } catch (err) {
    return {
      label: "Lighter alternatives",
      status: "error",
      count: 0,
      data: [],
      summary: `Could not read package.json: ${err.message}`,
    };
  }
}
```

```
// File: src\deps\unusedPackages.js
import depcheck from "depcheck";
import { loadConfig } from "../utils/config.js";

export async function checkUnusedPackages() {
  const config = await loadConfig();
  const ignore = config.scan?.ignore ?? [];

  try {
    const result = await depcheck(process.cwd(), {
      ignoreDirs: ignore.map((p) => p.replace(/\/$/, "")),
      skipMissing: true,
    });

    const unused = [...result.dependencies, ...result.devDependencies];

    return {
      label: "Unused packages",
      status: unused.length > 0 ? "warn" : "ok",
      count: unused.length,
      data: unused,
      summary:
        unused.length > 0
          ? `${unused.length} unused package(s) found`
          : "No unused packages",
    };
  } catch (err) {
    return {
      label: "Unused packages",
      status: "error",
      count: 0,
      data: [],
      summary: `depcheck failed: ${err.message}`,
    };
  }
}
```

```
// File: src\index.js
export { runScan } from "./commands/scan.js";
export { runDeps } from "./commands/deps.js";
export { runScore } from "./commands/score.js";
export { default as cleanCommand } from "./commands/clean.js";
export { default as securityCommand } from "./commands/security.js";
```

```
// File: src\scanners\.gitkeep
```

```
// File: src\scanners\consoleLogs.js
import fs from "fs";
import { walkFiles } from "../utils/fileWalker.js";
import { loadConfig } from "../utils/config.js";

export async function scanConsoleLogs() {
  const config = await loadConfig();
  const files = walkFiles(
    process.cwd(),
    config.scan.extensions,
    config.scan.ignore,
  );

  const found = [];

  for (const file of files) {
    const content = fs.readFileSync(file, "utf-8");
    const lines = content.split("\n");

    lines.forEach((line, index) => {
      if (/console\.(log|warn|error|info|debug)\s*\(/.test(line)) {
        found.push({
          file: file.replace(process.cwd() + "/", ""),
          line: index + 1,
          code: line.trim(),
        });
      }
    });
  }

  return {
    label: "Console Logs",
    status: found.length > 0 ? "warn" : "ok",
    count: found.length,
    data: found.map((f) => `${f.file}:${f.line}  ${f.code}`),
    summary:
      found.length > 0
        ? `${found.length} console.log(s) found`
        : "No console logs found",
  };
}
```

```
// File: src\scanners\duplicateCode.js
import { execSync } from "child_process";
import fs from "fs";
import path from "path";
import { loadConfig } from "../utils/config.js";

export async function scanDuplicateCode() {
  const config = await loadConfig();
  const reportDir = path.join(process.cwd(), ".boost-temp");
  const reportFile = path.join(reportDir, "jscpd-report.json");

  try {
    const srcDir = path.join(process.cwd(), "src");
    const scanTarget = fs.existsSync(srcDir) ? srcDir : process.cwd();

    execSync(
      `npx jscpd "${scanTarget}" --min-lines 5 --min-tokens 50 --reporters
json --silent --output "${reportDir}"`,
      { encoding: "utf-8", stdio: ["pipe", "pipe", "ignore"] },
    );

    let duplicates = [];
    if (fs.existsSync(reportFile)) {
      const json = JSON.parse(fs.readFileSync(reportFile, "utf-8"));
      duplicates = json.duplicates || [];
    }

    return {
      label: "Duplicate Code",
      status: duplicates.length > 0 ? "warn" : "ok",
      count: duplicates.length,
      data: duplicates.map((d) => {
        const fileA = d.firstFile?.name?.replace(process.cwd() + "/", "") ||
"";
        const fileB =
          d.secondFile?.name?.replace(process.cwd() + "/", "") || "";
        return `${fileA} !” ${fileB}`;
      }),
      summary:
        duplicates.length > 0
          ? `${duplicates.length} duplicate block(s) found`
          : "No duplicate code found",
    };
  } catch {
    return {
      label: "Duplicate Code",
      status: "ok",
      count: 0,
      data: [],
      summary: "Could not run duplicate check",
    };
  } finally {
    // always clean up temp folder after reading
    if (fs.existsSync(reportDir)) {
      fs.rmSync(reportDir, { recursive: true, force: true });
    }
  }
}
```

```
// File: src\scanners\largeFiles.js
import fs from "fs";
import { walkFiles } from "../utils/fileWalker.js";
import { loadConfig } from "../utils/config.js";

function parseThreshold(threshold) {
  const units = { kb: 1024, mb: 1024 * 1024 };
  const match = threshold.toLowerCase().match(/^(\d+)(kb|mb)$/);
  if (!match) return 500 * 1024;
  return parseInt(match[1]) * units[match[2]];
}

function formatSize(bytes) {
  if (bytes >= 1024 * 1024) return (bytes / (1024 * 1024)).toFixed(2) + " MB";
  return (bytes / 1024).toFixed(2) + " KB";
}

export async function scanLargeFiles() {
  const config = await loadConfig();
  const threshold = parseThreshold(config.scan.largeFileThreshold);
  const files = walkFiles(process.cwd(), [], config.scan.ignore);

  const found = [];

  for (const file of files) {
    const stat = fs.statSync(file);
    if (stat.size > threshold) {
      found.push({
        file: file.replace(process.cwd() + "/", ""),
        size: formatSize(stat.size),
        bytes: stat.size,
      });
    }
  }

  found.sort((a, b) => b.bytes - a.bytes);

  return {
    label: "Large Files",
    status: found.length > 0 ? "warn" : "ok",
    count: found.length,
    data: found.map((f) => `${f.file} (${f.size})`),
    summary:
      found.length > 0
        ? `${found.length} file(s) above ${config.scan.largeFileThreshold}`
        : "No large files found",
  };
}
```

```
// File: src\scanners\unusedAssets.js
import fs from "fs";
import path from "path";
import { walkFiles } from "../utils/fileWalker.js";
import { loadConfig } from "../utils/config.js";

const ASSET_EXTENSIONS = [
  ".png",
  ".jpg",
  ".jpeg",
  ".gif",
  ".svg",
  ".webp",
  ".ico",
  ".ttf",
  ".woff",
  ".woff2",
  ".eot",
];

export async function scanUnusedAssets() {
  const config = await loadConfig();
  const allFiles = walkFiles(process.cwd(), [], config.scan.ignore);

  const assetFiles = allFiles.filter((f) =>
    ASSET_EXTENSIONS.includes(path.extname(f).toLowerCase()),
  );
  const codeFiles = allFiles.filter((f) =>
    config.scan.extensions.includes(path.extname(f)),
  );

  const codeContent = codeFiles
    .map((f) => fs.readFileSync(f, "utf-8"))
    .join("\n");

  const unused = assetFiles.filter((assetPath) => {
    const assetName = path.basename(assetPath);
    return !codeContent.includes(assetName);
  });

  return {
    label: "Unused Assets",
    status: unused.length > 0 ? "warn" : "ok",
    count: unused.length,
    data: unused.map((f) => f.replace(process.cwd() + "/", "")),
    summary:
      unused.length > 0
        ? `${unused.length} unused asset(s) found`
        : "No unused assets found",
  };
}
```

```
// File: src\scanners\unusedFiles.js
import fs from "fs";
import path from "path";
import { walkFiles } from "../utils/fileWalker.js";
import { loadConfig } from "../utils/config.js";

function getImportsFromFile(filePath) {
  const content = fs.readFileSync(filePath, "utf-8");
  const importRegex = /from\s+['"]([^'"]+)['"]/g;
  const requireRegex = /require\s*\(\s*['"]([^'"]+)['"]\s*\)/g;
  const imports = new Set();

  let match;
  while ((match = importRegex.exec(content)) !== null) imports.add(match[1]);
  while ((match = requireRegex.exec(content)) !== null) imports.add(match[1]);

  return imports;
}

function resolveImport(importPath, fromFile, allFiles) {
  if (!importPath.startsWith(".")) return null;

  const dir = path.dirname(fromFile);
  const resolved = path.resolve(dir, importPath);

  const extensions = [
    ".js",
    ".ts",
    ".jsx",
    ".tsx",
    ".vue",
    "/index.js",
    "/index.ts",
  ];
  for (const ext of extensions) {
    const candidate = resolved + ext;
    if (allFiles.includes(candidate)) return candidate;
  }

  if (allFiles.includes(resolved)) return resolved;
  return null;
}

export async function scanUnusedFiles() {
  const config = await loadConfig();

  // get entry point from package.json
  let entryPoint = null;
  try {
    const pkg = JSON.parse(
      fs.readFileSync(path.resolve(process.cwd(), "package.json"), "utf-8"),
    );
    entryPoint = pkg.main ? path.resolve(process.cwd(), pkg.main) : null;
  } catch {}

  const allFiles = walkFiles(
    process.cwd(),
    config.scan.extensions,
    config.scan.ignore,
  );
```

```
  if (!entryPoint || !fs.existsSync(entryPoint)) {
    return {
      label: "Unused Files",
      status: "warn",
      count: 0,
      data: [],
      summary: "Could not detect entry point from package.json main",
    };
  }

  const visited = new Set();

  function traverse(filePath) {
    if (visited.has(filePath)) return;
    visited.add(filePath);

    const imports = getImportsFromFile(filePath);
    for (const imp of imports) {
      const resolved = resolveImport(imp, filePath, allFiles);
      if (resolved) traverse(resolved);
    }
  }

  traverse(entryPoint);

  const unused = allFiles.filter((f) => !visited.has(f));

  return {
    label: "Unused Files",
    status: unused.length > 0 ? "warn" : "ok",
    count: unused.length,
    data: unused.map((f) => f.replace(process.cwd() + "/", "")),
    summary:
      unused.length > 0
        ? `${unused.length} unused file(s) found`
        : "No unused files found",
  };
}
```

```
// File: src\scorer\.gitkeep
```

```
// File: src\scorer\healthScore.js
import { checkUnusedPackages } from "../deps/unusedPackages.js";
import { checkHeavyPackages } from "../deps/heavyPackages.js";
import { checkLighterAlternatives } from "../deps/lighterAlternatives.js";
import envCommitted from "../security/envCommitted.js";
import hardcodedKeys from "../security/hardcodedKeys.js";
import auditWrapper from "../security/auditWrapper.js";
import tempFiles from "../cleaners/tempFiles.js";
import emptyFolders from "../cleaners/emptyFolders.js";
import { scanUnusedFiles } from "../scanners/unusedFiles.js";
import { scanLargeFiles } from "../scanners/largeFiles.js";
import { scanDuplicateCode } from "../scanners/duplicateCode.js";
import { scanUnusedAssets } from "../scanners/unusedAssets.js";
import { scanConsoleLogs } from "../scanners/consoleLogs.js";

// % % %  Scoring config % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
//
// Each check has:
//   maxPts     — full marks if zero issues
//   deductPer  — points deducted per issue found
//   errorPts   — score assigned if the check itself errors out
//
// Security is weighted heaviest — a single committed .env wipes its check.
// Scan is second — code quality matters.
// Deps is third — dependency hygiene.
// Clean is lightest — cosmetic, not critical.
//
const CHECKS = {
  // Scan — 40 pts total
  unusedFiles: { maxPts: 10, deductPer: 2, errorPts: 5 },
  largeFiles: { maxPts: 8, deductPer: 2, errorPts: 4 },
  duplicateCode: { maxPts: 10, deductPer: 3, errorPts: 5 },
  unusedAssets: { maxPts: 6, deductPer: 1, errorPts: 3 },
  consoleLogs: { maxPts: 6, deductPer: 1, errorPts: 3 },

  // Deps — 20 pts total
  unused: { maxPts: 8, deductPer: 2, errorPts: 4 },
  heavy: { maxPts: 6, deductPer: 2, errorPts: 3 },
  alternatives: { maxPts: 6, deductPer: 1, errorPts: 3 },

  // Security — 30 pts total
  envCommitted: { maxPts: 12, deductPer: 12, errorPts: 0 },
  hardcodedKeys: { maxPts: 12, deductPer: 4, errorPts: 0 },
  auditVulns: { maxPts: 6, deductPer: 2, errorPts: 3 },

  // Clean — 10 pts total
  tempFiles: { maxPts: 5, deductPer: 1, errorPts: 2 },
  emptyFolders: { maxPts: 5, deductPer: 1, errorPts: 2 },
};

const CATEGORIES = {
  scan: {
    label: "Code Quality",
    keys: [
      "unusedFiles",
      "largeFiles",
      "duplicateCode",
      "unusedAssets",
      "consoleLogs",
    ],
    maxPts: 40,
```

```
  },
  security: {
    label: "Security",
    keys: ["envCommitted", "hardcodedKeys", "auditVulns"],
    maxPts: 30,
  },
  deps: {
    label: "Dependencies",
    keys: ["unused", "heavy", "alternatives"],
    maxPts: 20,
  },
  clean: {
    label: "Cleanliness",
    keys: ["tempFiles", "emptyFolders"],
    maxPts: 10,
  },
};

// % % %  Helpers % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %

function scoreCheck(result, check) {
  if (result.status === "error") return check.errorPts;
  const deducted = result.count * check.deductPer;
  return Math.max(0, check.maxPts - deducted);
}

function letterGrade(score) {
  if (score >= 90) return "A";
  if (score >= 80) return "B";
  if (score >= 70) return "C";
  if (score >= 50) return "D";
  return "F";
}

// % % %  Main export % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %

export async function computeHealthScore() {
  // run all checks in parallel
  const [
    unusedFilesResult,
    largeFilesResult,
    duplicateCodeResult,
    unusedAssetsResult,
    consoleLogsResult,
    unused,
    heavy,
    alternatives,
    envResult,
    keysResult,
    auditResult,
    tempResult,
    foldersResult,
  ] = await Promise.all([
    scanUnusedFiles(),
    scanLargeFiles(),
    scanDuplicateCode(),
    scanUnusedAssets(),
    scanConsoleLogs(),
    checkUnusedPackages(),
    checkHeavyPackages(),
    checkLighterAlternatives(),
```

```
  envCommitted(),
  hardcodedKeys(),
  auditWrapper(),
  tempFiles(),
  emptyFolders(),
]);

const allResults = {
  // scan
  unusedFiles: unusedFilesResult,
  largeFiles: largeFilesResult,
  duplicateCode: duplicateCodeResult,
  unusedAssets: unusedAssetsResult,
  consoleLogs: consoleLogsResult,
  // deps
  unused,
  heavy,
  alternatives,
  // security
  envCommitted: envResult,
  hardcodedKeys: keysResult,
  auditVulns: auditResult,
  // clean
  tempFiles: tempResult,
  emptyFolders: foldersResult,
};

// score each individual check
const checkScores = {};
for (const [key, check] of Object.entries(CHECKS)) {
  checkScores[key] = {
    pts: scoreCheck(allResults[key], check),
    maxPts: check.maxPts,
    result: allResults[key],
  };
}

// roll up into categories
const categoryScores = {};
for (const [catKey, cat] of Object.entries(CATEGORIES)) {
  const earned = cat.keys.reduce((sum, k) => sum + checkScores[k].pts, 0);
  categoryScores[catKey] = {
    label: cat.label,
    earned,
    maxPts: cat.maxPts,
    pct: Math.round((earned / cat.maxPts) * 100),
  };
}

const totalEarned = Object.values(categoryScores).reduce(
  (s, c) => s + c.earned,
  0,
);
const totalMax = Object.values(categoryScores).reduce(
  (s, c) => s + c.maxPts,
  0,
);
const score = Math.round((totalEarned / totalMax) * 100);

return {
  score,
```

```
    grade: letterGrade(score),
    categories: categoryScores,
    checks: checkScores,
    results: allResults,
  };
}
```

```
// File: src\security\.gitkeep
```

```
// File: src\security\auditWrapper.js
import { exec } from "child_process";
import { promisify } from "util";

const execAsync = promisify(exec);

export default async function auditWrapper() {
  try {
    const { stdout } = await execAsync("npm audit --json");

    const audit = JSON.parse(stdout);

    const vulnerabilities = audit.metadata?.vulnerabilities?.total || 0;

    return {
      label: "npm Vulnerabilities",
      status: vulnerabilities ? "warn" : "ok",
      count: vulnerabilities,
      data: [],
      summary: vulnerabilities
        ? `${vulnerabilities} vulnerabilities found`
        : "No vulnerabilities found",
    };
  } catch (err) {
    return {
      label: "npm Vulnerabilities",
      status: "error",
      count: 0,
      data: [],
      summary: "npm audit failed",
    };
  }
}
```

```
// File: src\security\envCommitted.js
import fg from "fast-glob";
import path from "path";
import { loadConfig } from "../utils/config.js";

export default async function envCommitted() {
  try {
    const config = await loadConfig();

    const files = await fg(["**/.env*", "!node_modules/**"], {
      dot: true,
      absolute: true,
    });

    const found = files.map((f) => path.relative(process.cwd(), f));

    return {
      label: ".env Files",
      status: found.length ? "warn" : "ok",
      count: found.length,
      data: found,
      summary: found.length
        ? `${found.length} .env file(s) detected`
        : "No committed .env files found",
    };
  } catch (err) {
    return {
      label: ".env Files",
      status: "error",
      count: 0,
      data: [],
      summary: "Failed to scan for .env files",
    };
  }
}
```

```
// File: src\security\hardcodedKeys.js
import fg from "fast-glob";
import fs from "fs/promises";
import path from "path";
import { loadConfig } from "../utils/config.js";

const SECRET_PATTERNS = [
  /api[_-]?key\s*=\s*['"][A-Za-z0-9-_]{16,}['"]/i,
  /secret\s*=\s*['"][A-Za-z0-9-_]{16,}['"]/i,
  /token\s*=\s*['"][A-Za-z0-9-_]{16,}['"]/i,
  /password\s*=\s*['"][^'"]{6,}['"]/i,
];

export default async function hardcodedKeys() {
  try {
    const config = await loadConfig();

    const files = await fg(config.security.scanPatterns, {
      ignore: ["node_modules/**"],
      absolute: true,
    });

    const findings = [];

    for (const file of files) {
      const content = await fs.readFile(file, "utf8");

      for (const pattern of SECRET_PATTERNS) {
        if (pattern.test(content)) {
          findings.push(path.relative(process.cwd(), file));
          break;
        }
      }
    }

    return {
      label: "Hardcoded Secrets",
      status: findings.length ? "warn" : "ok",
      count: findings.length,
      data: findings,
      summary: findings.length
        ? `${findings.length} potential secret(s) found`
        : "No hardcoded secrets detected",
    };
  } catch (err) {
    return {
      label: "Hardcoded Secrets",
      status: "error",
      count: 0,
      data: [],
      summary: "Secret scan failed",
    };
  }
}
```

```
// File: src\utils\.gitkeep
```

```
// File: src\utils\config.js
import { createRequire } from "module";
import { pathToFileURL } from "url";
import path from "path";
import fs from "fs";

const defaults = {
  scan: {
    largeFileThreshold: "500kb",
    ignore: ["dist/", "node_modules/", ".git/", "coverage/"],
    extensions: [".js", ".ts", ".jsx", ".tsx", ".vue"],
  },
  clean: {
    tempPatterns: ["**/*.tmp", "**/*.cache", "**/.DS_Store", "**/Thumbs.db"],
  },
  deps: {
    checkBundleSize: true,
  },
  security: {
    scanPatterns: ["**/*.js", "**/*.ts", "**/*.jsx", "**/*.tsx", "**/*.env*"],
  },
};

export async function loadConfig() {
  const configPath = path.resolve(process.cwd(), "boost.config.js");
  if (fs.existsSync(configPath)) {
    const userConfig = await import(pathToFileURL(configPath).href);
    return deepMerge(defaults, userConfig.default);
  }
  return defaults;
}

function deepMerge(base, override) {
  const result = { ...base };
  for (const key of Object.keys(override)) {
    if (typeof override[key] === "object" && !Array.isArray(override[key])) {
      result[key] = deepMerge(base[key] || {}, override[key]);
    } else {
      result[key] = override[key];
    }
  }
  return result;
}
```

```
// File: src\utils\fileWalker.js
import fs from "fs";
import path from "path";
import { loadGitignore } from "./gitignoreParser.js";

export function walkFiles(dir, extensions = [], extraIgnore = []) {
  const ig = loadGitignore(process.cwd());
  ig.add(extraIgnore);

  const results = [];

  function walk(currentDir) {
    const entries = fs.readdirSync(currentDir, { withFileTypes: true });

    for (const entry of entries) {
      const fullPath = path.join(currentDir, entry.name);
      const relativePath = path.relative(process.cwd(), fullPath);

      if (ig.ignores(relativePath)) continue;

      if (entry.isDirectory()) {
        walk(fullPath);
      } else if (entry.isFile()) {
        const ext = path.extname(entry.name);
        if (extensions.length === 0 || extensions.includes(ext)) {
          results.push(fullPath);
        }
      }
    }
  }

  walk(dir);
  return results;
}
```

```
// File: src\utils\gitignoreParser.js
import fs from "fs";
import path from "path";
import ignore from "ignore";

export function loadGitignore(cwd = process.cwd()) {
  const ig = ignore();
  const gitignorePath = path.join(cwd, ".gitignore");
  if (fs.existsSync(gitignorePath)) {
    const content = fs.readFileSync(gitignorePath, "utf-8");
    ig.add(content);
  }
  return ig;
}
```

```
// File: src\utils\logger.js
import chalk from "chalk";

export const logger = {
  info: (msg) => console.log(chalk.cyan("!9"), msg),
  success: (msg) => console.log(chalk.green("' "), msg),
  warn: (msg) => console.log(chalk.yellow("& "), msg),
  error: (msg) => console.log(chalk.red("' "), msg),
  title: (msg) => console.log("\n" + chalk.bold.white(msg)),
  dim: (msg) => console.log(chalk.dim(msg)),
  blank: () => console.log(""),

  result: (result) => {
    if (result.status === "ok") {
      logger.success(
        chalk.green(result.label) + chalk.dim(` — ${result.summary}`),
      );
    } else if (result.status === "warn") {
      logger.warn(
        chalk.yellow(result.label) + chalk.dim(` — ${result.summary}`),
      );
      if (result.data?.length) {
        result.data.forEach((item) => {
          console.log(
            chalk.dim("   !’"),
            chalk.white(typeof item === "string" ? item :
JSON.stringify(item)),
          );
        });
      }
    } else if (result.status === "error") {
      logger.error(chalk.red(result.label) + chalk.dim(` — ${result.summary}
`));
    }
  },
};
```

```
// File: src\utils\spinner.js
import ora from "ora";

let activeSpinner = null;

export function startSpinner(text) {
  activeSpinner = ora({ text, color: "cyan" }).start();
  return activeSpinner;
}

export function stopSpinner(success = true, text = "") {
  if (!activeSpinner) return;
  if (success) {
    activeSpinner.succeed(text || activeSpinner.text);
  } else {
    activeSpinner.fail(text || activeSpinner.text);
  }
  activeSpinner = null;
}

export function updateSpinner(text) {
  if (activeSpinner) activeSpinner.text = text;
}
```